

A Control Perspective on Training PINNs [★]

Matthieu Barreau ^{*} Haoming Shen ^{*}

^{*} *Digital Futures and KTH Royal Institute of Technology, Stockholm, Sweden (e-mail: barreau,haomings@kth.se).*

Abstract: We investigate the training of Physics-Informed Neural Networks (PINNs) from a control-theoretic perspective. Using gradient descent with resampling, we interpret the training dynamics as asymptotically equivalent to a stochastic control-affine system, where sampling effects act as process disturbances and measurement noise. Within this framework, we introduce two controllers for dynamically adapting the physics weight: an integral controller and a leaky integral controller. We theoretically analyze their asymptotic properties under the accuracy–robustness trade-off and we evaluate them on a toy example. Numerical evidence suggests that the integral controller achieves accurate and robust convergence when the physical model is correct, whereas the leaky integrator provides improved performance in the presence of model mismatch. This work represents a first step toward convergence guarantees and principled training algorithms tailored to the distinct characteristics of PINN tasks.

1. INTRODUCTION

Physics-Informed Neural Networks (PINNs) have emerged as a powerful deep learning paradigm for learning solutions to differential equations and characterizing complex dynamical behaviors (Karniadakis et al., 2021). The key advantages lie in the incorporation of physical laws into the minimization of the loss function, which can be interpreted as a regularization agent that enforces physical constraints via penalty terms, enabling efficient learning even in the presence of limited available data. However, like any deep learning methods, PINNs inherit stochastic properties from their underlying architecture. Moreover, PINNs are formulated as multi-objective optimization problems that involve multiple loss components (e.g., losses associated with physical residuals, initial conditions, and boundary conditions), in which unbalanced gradient flow can lead to instability during training (Wang et al., 2021). These issues pose barriers to achieving robust and efficient training, particularly for large-scale or complex systems.

Improving the accuracy and robustness of neural network training has attracted significant attention in the Machine Learning community. A successful example is the Xavier initialization, which has been shown to substantially improve training stability, particularly for large and deep neural networks (Glorot and Bengio, 2010). Meanwhile, various optimization algorithms such as Adam have demonstrated strong empirical performance in accelerating convergence and enhancing robustness (Kingma and Ba, 2014). In addition, growing research interest has focused on developing methods specifically tailored to improve PINNs performance, including pretraining (Guo et al., 2023; Li et al., 2025), reformulations of the underlying mathematical problem (Sirignano and Spiliopoulos, 2018; Son et al., 2023), novel architectures (Shukla et al., 2021; Jagtap et al., 2020), and new learning paradigms

such as meta-learning and curriculum learning (Psaros et al., 2022; Liu et al., 2024).

One line of studies focuses on adaptively adjusting the weights associated with each loss term to mitigate gradient imbalance and numerical stiffness during training. Successful implementations include rescaling the data loss relative to the physics loss based on observed gradient magnitudes (Wang et al., 2021), or utilizing neural tangent kernel analysis for PINNs to ensure similar convergence rates for all losses (Wang et al., 2022). Maddu et al. (2022) propose a variance-based reweighting scheme by monitoring the variance of gradients and assigning inversely to their uncertainty, which helps prevent gradient vanishing. By introducing trainable weights, PINNs can adaptively penalize hard-to-minimize loss components during the training process (McClenny and Braga-Neto, 2023).

Despite existing reweighting strategies that improve PINNs’ performance, the training process can exhibit significant variability and sensitivity to uncertainty due to the intrinsic stochastic optimization dynamics inherent in batch training, initialization, and the complex architectures of PINNs. Moreover, the proposed approaches are empirical and often lack interpretations. In contrast, a rich body of research exists in the control community, focusing on ensuring stability and robustness in system dynamics under uncertainty.

Motivated by Cerone et al. (2025), where the learning under constraints problem is framed as a dynamical system, our contribution is to reinterpret PINN training as a control-affine system and analyze controller choices. In Section 2, we introduce some background and formulate the problem. Then, we propose investigating certain properties of the optimization problem to establish the existence of a solution and characterize the equilibrium points in Section 3. Section 4 describes the gradient descent algorithm in the form of a noisy dynamical system. Section 5 presents the main contribution, where we study two basic control schemes: an integral control for achieving

[★] This work was partially supported by the Wallenberg AI, Autonomous Systems and Software Program (WASP) funded by the Knut and Alice Wallenberg Foundation.

zero steady-state error and a leaky integrator for enhanced robustness. Finally, we investigate a toy example in Section 6 to showcase the potential of the approach. Section 7 concludes the work with a summary and perspectives.

Notations: We use $(\mathbb{R}^n, \|\cdot\|)$ as the real normed vector space. For a finite set $\Omega \subset \mathbb{R}^n$, $|\Omega|$ refers to its cardinal, and in the case of a compact set, $|\Omega|$ is its volume. We define the following functional spaces: $\mathcal{L}^2(A, B)$ as the space of squared integrable functions from A to B , $\mathcal{H}^q(A, B) \subseteq \mathcal{L}^2(A, B)$ is the Sobolev space of functions from A to B which have weak-derivatives up to the order q . If A is a compact subset of $\mathbb{R}^n$, the norm used in $\mathcal{H}^q(A, B)$ is

$$\|v\|_{\mathcal{H}^q}^2 = \sum_{k=0}^q \sum_{|\alpha|=k} \int_A \|D^\alpha v(u)\|^2 du$$

where $\alpha = (\alpha_1, \dots, \alpha_n)$ is a multi-index of order $|\alpha| = k$ and $D^\alpha f = \frac{\partial^{|\alpha|} f}{\partial x_1^{\alpha_1} \dots \partial x_n^{\alpha_n}}$.

2. PROBLEM FORMULATION

Let $\mathfrak{F} : \mathcal{H}^q(U, \mathbb{R}^n) \rightarrow \mathcal{L}^2(U, \mathbb{R}^n)$ be a possibly nonlinear operator where U is an open bounded subset of $\mathbb{R}^p$. We consider the ODE/PDE problem

$$\begin{aligned} \mathfrak{F}[v](u) &= 0, \quad u \in U, \\ v(s) &= v_\Gamma(s), \quad s \in \Gamma_1 \cup \Gamma_2, \end{aligned} \quad (1)$$

where $v : U \rightarrow \mathbb{R}^n$ belongs to a Sobolev space $\mathcal{H}^q(U, \mathbb{R}^n)$ with $q \geq 0$, Γ_1 is a subset of the boundary of U and v_Γ is prescribed as the initial and boundary conditions. Additional in-domain point-wise knowledge is provided through $\Gamma_2 \subset U$.

Example 1. Consider the following autonomous ordinary differential equation:

$$\dot{x}(t) = f(x(t)), \quad t \geq 0, \quad (2)$$

with $x(0) = x_0 \in \mathbb{R}^n$. This fits the framework in (1) with $v = x$, $q = 1$, $U = [0, T]$ for any $T > 0$, $\Gamma_1 = \{0\}$ with $v_\Gamma(0) = x_0$, and

$$\forall u \in [0, T], \quad \mathfrak{F}[v](u) = \dot{v}(u) - f(v(u)). \quad \triangle$$

Example 2. Consider the following second-order partial differential equation:

$$\begin{aligned} \partial_t v(t, x) &= -\gamma(t, x, v(t, x), \nabla_x v(t, x)) \\ &\quad + \operatorname{div}(\alpha(t, x, v(t, x), \nabla_x v(t, x))) \end{aligned} \quad (3)$$

for $t, x \in [0, T] \times \Omega \subset \mathbb{R}^n$ with $v(0, \cdot) = v_0$, $v(t, x) = g(t, x)$ for $t \in (0, T]$ and $x \in \partial\Omega$. We can define the operator $\mathfrak{F}[v] = \partial_t v + \gamma - \operatorname{div} \alpha$ and $U = (0, T] \times \Omega$. $\triangle$

Assuming there is a unique weak-solution v to (1) with $\Gamma_2 = \emptyset$, the *forward* problem in PINNs is to approximate v using a deep feed-forward neural network v_Θ with L hidden layers and a linear output defined by

$$v_\Theta(u) = W_{L+1} \times H_L \circ \dots \circ H_1(u) + b_{L+1}, \quad u \in U \quad (4)$$

where $H_i(u) = \phi(W_i u + b_i)$, $\phi \in \mathcal{C}^\infty(\mathbb{R}, \mathbb{R})$ being an element-wise activation function, b_i and W_i are real matrices of appropriate dimensions. The number of neurons in each layer is denoted $N \in \mathbb{N}$. The function v_Θ is parametrized by the tensor $\Theta = \{b_1, W_1, \dots, b_{L+1}, W_{L+1}\}$. The *inverse* problem is an identification task where we try to learn $\mathfrak{F}$ such that $v_\Theta - v_\Gamma$ is also minimized on Γ_2 .

Approximating a weak solution to the original problem (1) is equivalent to solving the following one on v_Θ :

$$\begin{aligned} \Theta_* &\in \underset{\Theta}{\operatorname{Argmin}} \int_\Gamma \|v_\Gamma(s) - v_\Theta(s)\|^2 ds \\ \text{s.t.} \quad &\int_U \|\mathfrak{F}[v_\Theta](u)\|^2 du = 0. \end{aligned} \quad (5)$$

The vanilla PINN framework (Karniadakis et al., 2021) discretizes the sets Γ and U into D_Γ and D_U and then creates the data and physics costs as

$$\begin{aligned} \mathcal{L}_{\text{data}}(\Theta, D_\Gamma) &= \frac{1}{|D_\Gamma|} \sum_{s \in D_\Gamma} \|v(s) - v_\Theta(s)\|^2, \\ \mathcal{L}_{\text{phys}}(\Theta, D_U) &= \frac{1}{|D_U|} \sum_{u \in D_U} \|\mathfrak{F}[v_\Theta](u)\|^2. \end{aligned}$$

The data cost is the mean squared error between the measurement points and the neural network v_Θ evaluated at the same locations, and the physics one is the mean squared error between $\mathfrak{F}[v_\Theta]$ and zero on the discrete set D_U . By considering the physics cost as a regularization agent, it leads to the extended discretized cost:

$$\mathcal{L}_\lambda^d(\Theta, D_\Gamma, D_U) = \mathcal{L}_{\text{data}}(\Theta, D_\Gamma) + \lambda \mathcal{L}_{\text{phys}}(\Theta, D_U) \quad (6)$$

where $\lambda > 0$, and (5) is relaxed to

$$\Theta_* = \underset{\Theta}{\operatorname{Argmin}} \mathcal{L}_\lambda^d(\Theta, D_\Gamma, D_U). \quad (7)$$

Remark 1. Technically speaking, if $\mathfrak{F}[v_\Theta]$ contains derivatives of v_Θ , one can use automatic differentiation to calculate it appropriately (Baydin et al., 2017). $\blacklozenge$

A strategy to approximate a solution to (7) is to use a gradient-descent algorithm, which writes as:

$$\Theta_{k+1} = \Theta_k - \alpha(k) \nabla_\Theta \mathcal{L}_\lambda^d(\Theta_k, D_\Gamma, D_U), \quad \Theta(0) \sim G \quad (8)$$

where $\alpha > 0$ is the learning rate and G is the Xavier-Glorot initialization (Glorot and Bengio, 2010). Under some conditions on the loss $\mathcal{L}_\lambda$ and the learning rate α , it is possible to prove convergence of (8). In these cases, Θ has a limit Θ_* and $\nabla_\Theta \mathcal{L}_\lambda^d(\Theta_*) = 0$. That means the algorithm reached a local optimum. However, this local optimum might be far from the global optimum, and the following generalization error might be large:

$$\mathcal{E}(\Theta) = \int_U \|v(u) - v_\Theta(u)\|^2 du. \quad (9)$$

Therefore, a training algorithm $\mathcal{T}$ is evaluated over two quantities:

$$A(\mathcal{T}) = \mathbb{E}_{\Theta_0 \sim G} \{\mathcal{E}(\mathcal{T}(\Theta_0))\}, \quad R(\mathcal{T}) = \mathbb{V}_{\Theta_0 \sim G} \{\mathcal{E}(\mathcal{T}(\Theta_0))\}.$$

The stochasticity of the training algorithm, due to its internal processes and the initial condition, necessitates considering the average and variance of the error as accuracy A and robustness R metrics, respectively. Increasing the robustness of a training algorithm (small R) is desirable to ensure that its accuracy accurately reflects its efficiency.

In the case $\Gamma_2 \neq \emptyset$, there are two different regimes possible for the accuracy. Either $\|v_\Gamma - v_{\Theta_*}\|$ on Γ_2 is minimal; this is the *match* case. Alternatively, there may be a discrepancy, and the additional measurements contradict the physical model; this is the *mismatch* case. Solving the forward problem in both these cases is the setup of this paper.

Problem Statement: We want to design a robust, accurate, and simple algorithm for training a PINN in the match and mismatch cases.

3. PRELIMINARIES

All the above definitions assume that there is a unique solution to compare with, and that (5) admits a global minimum. To ensure these two key properties, we make the following assumptions.

Assumption 1. We assume that problem (1) is well-posed with $\Gamma_2 = \emptyset$, meaning that there exists a unique solution $v : U \rightarrow \mathbb{R}^n$ which satisfies Equation (1) in a weak sense.

Assumption 2. The operator $\mathfrak{F}$ is globally Lipschitz continuous.

The first assumption is required to infer a solution, while the second one is a classical assumption for analysis. Note that, even if very conservative, the second assumption can be relaxed to local Lipschitz continuity if the solution stays bounded (Hartman, 2002).

Example 3. (Ex.1 continued). Let $\mathfrak{F}[v](u) = \dot{v}(u) - f(x(u))$ and assume that f is globally Lipschitz continuous with constant L_f . Then, by Picard-Lindelöf existence theorem, there exists a unique solution $v \in \mathcal{H}^1(\mathbb{R}^+, \mathbb{R}^n)$ to (2). Therefore, Assumption 1 is verified.

Moreover, we get that for all $v, \hat{v} \in \mathcal{H}^1$:

$$\begin{aligned} & \int_0^T \|\mathfrak{F}[\hat{v}](t) - \mathfrak{F}[v](t)\|^2 dt \\ & \leq 2 \int_0^T \left(\|\dot{v}(t) - \dot{\hat{v}}(t)\|^2 + \|f(\hat{v}(t)) - f(v(t))\|^2 \right) dt \\ & \leq 2 \int_0^T \left(\|\dot{v}(t) - \dot{\hat{v}}(t)\|^2 + L_f^2 \|\hat{v}(t) - v(t)\|^2 \right) dt \\ & \leq 2 \max(1, L_f^2) \|\hat{v} - v\|_{\mathcal{H}^1}^2. \end{aligned}$$

Consequently, $\mathfrak{F}$ is a globally Lipschitz operator, and Assumption 2 is verified. $\triangle$

Example 4. (Ex.2 continued). Let $\mathfrak{F}[v] = \partial_t v + \gamma - \operatorname{div} \alpha$ and $U = (0, T] \times \Omega$. Then, under some conditions on the domain Ω , and on the functions α and γ , there exists a unique solution to (3) (Ladyzhenskaia et al., 1968). Moreover, under some other conditions (such as polynomial growth) on α and γ , the operator $\mathfrak{F}$ is a global Lipschitz operator (Sirignano and Spiliopoulos, 2018, Theorem 7.1). In that case, Assumptions 1 and 2 are verified. $\triangle$

The two assumptions are made to ensure that the neural-network approximation v_Θ proposed in (4) yields a global minimum loss of zero. Using the Lagrangian formulation, an optimal solution to (5) would be a minimizer of the following loss

$$\mathcal{L}_\lambda(\Theta) = \int_\Gamma \|v_\Gamma(s) - v_\Theta(s)\|^2 ds + \lambda \int_U \|\mathfrak{F}[v_\Theta](u)\|^2 du \quad (10)$$

with $\lambda > 0$.

Proposition 1. Assume that $L > 0, \Gamma_2 = \emptyset, \lambda > 0$, then the minimal value of $\mathcal{L}_\lambda$ can be made as small as desired, provided enough neurons N .

Proof. Assumption 1 guarantees that there exists $v \in \mathcal{H}^q$ such that $\mathfrak{F}[v] = 0$. From Assumption 2, there exists $L > 0$ such that

$$\forall v, \hat{v} \in \mathcal{H}^q, \quad \int_U \|\mathfrak{F}[\hat{v}](u) - \mathfrak{F}[v](u)\|^2 du \leq L^2 \|\hat{v} - v\|_{\mathcal{H}^q}^2.$$

Using the previous inequality and the fact that $v_\Theta \in \mathcal{H}^p$ as well, we get

$$\begin{aligned} \mathcal{L}_\lambda(\Theta) &= \int_\Gamma \|v_\Gamma(s) - v_\Theta(s)\|^2 ds \\ &\quad + \lambda \int_U \|\mathfrak{F}[v_\Theta](u) - \mathfrak{F}[v](u)\|^2 du \\ &\leq \int_\Gamma \|v_\Gamma(s) - v_\Theta(s)\|^2 ds + \lambda L^2 \|v_\Theta - v\|_{\mathcal{H}^p}^2. \end{aligned} \quad (11)$$

Under the conditions of the theorem, the universal approximation theorem by Hornik (1991) ensures that for any $\varepsilon > 0$, there exists a number of neurons $N > 0$ such that

$$\max \left(\sup_{s \in \Gamma} \|v_\Gamma(s) - v_\Theta(s)\|, \sup_{u \in U} \max_{|\alpha| \leq q} \|D^\alpha v(u)\| \right) < \varepsilon.$$

Plugging this back into (11) leads to the existence of $C > 0$ as a function of p and q such that

$$\mathcal{L}_\lambda(\Theta) \leq (|\Gamma| + C\lambda|U|L^2) \varepsilon^2$$

which ends the proof. $\square$

Note that, in some cases, if there exists Θ_* such that $\mathcal{L}_\lambda(\Theta_*) = 0$ then v_{Θ_*} is the unique solution to (1). This phenomenon occurs for certain classes of parabolic equations (Sirignano and Spiliopoulos, 2018, Theorem 7.3) and also for finite-dimensional dynamical systems, as demonstrated below.

Proposition 2. Assume the ODE case of Example 1. Then Proposition 1 holds and for $\lambda > 0$ and $|\Gamma| > 0$, $\mathcal{L}_\lambda(v) = 0$ is equivalent to v is the unique solution to (1).

Proof. Because Assumptions 1 and 2 hold, then Proposition 1 also holds. Moreover, $\mathcal{L}_\lambda(v) = 0$ implies that both contributions in $\mathcal{L}_\lambda$ must be zero and there exists $s \in \Gamma$ such that $v(s) = 0$ with $\mathfrak{F}[v] = 0$ almost everywhere. This is a Cauchy problem, and $v = v^*$ by the uniqueness property. $\square$

We have seen that if we find parameters Θ_* which are globally minimizing the Lagrangian $\mathcal{L}_\lambda$, then, provided some conditions on $\mathfrak{F}$, v_{Θ_*} is an approximation to v . This means that there is a low generalization error (9). The link between the generalization error and a small Lagrangian is provided. However, gradient-descent algorithms usually only find local optima. Some results in machine learning strongly indicate that local minima tend to converge to the global minimum, provided a large number of neurons (Choromanska et al., 2015). This suggests that using a large neural network would increase the accuracy and robustness of gradient-descent algorithms by decreasing the probability of reaching a "bad" local minimum. We will investigate strategies to strengthen this claim in the following sections.

4. CONTROL FORMULATION

The training algorithm proposed to estimate a solution to (5) is given in Algorithm 1. This is the standard gradient-descent procedure (Line 7) with additional steps such as the resampling sub-routines (Lines 3-6) together with an adaptive weight λ that is computed as the output of the function `compute_control`.

The dynamical system formulation of an algorithm is an effective way to analyze its asymptotic behavior. By setting $\xi = k\alpha$ and then making α go to 0. Under proper

Algorithm 1 Learning algorithm for (5).

```

1: Initialize  $\Theta_0 \sim G$  randomly,  $\lambda_0 = 0$ 
2: for  $k$  from 0 to  $N_{\text{epoch}}$  do
3:   if  $k \bmod N_s = 0$  then
4:     Sample  $N_\Gamma$  points uniformly in  $\Gamma$  to form  $D_\Gamma$ 
5:     Sample  $N_U$  points uniformly in  $U$  to form  $D_U$ 
6:   end if
7:    $\Theta_{k+1} = \Theta_k - \alpha(k) \nabla_{\Theta} \mathcal{L}_{\lambda_k}(\Theta_k, D_\Gamma, D_U)$ 
8:    $\lambda_{k+1} = \text{compute\_control}(\lambda_k, \Theta_k, D_U, D_\Gamma)$ 
9: end for
10: return  $\Theta_k$ 

```

conditions derived by Elkabetz and Cohen (2021), Line 7 of Algorithm 1 is equivalent to:

$$\frac{d}{d\xi} \Theta(\xi) = -\nabla_{\Theta} \mathcal{L}_{\lambda(\xi)}^d(\Theta(\xi), D_\Gamma, D_U). \quad (12)$$

We then leverage the framework proposed by Cerone et al. (2025) to reformulate the learning under constraints problem (5) together with the gradient descent scheme (12) as an input-output dynamical system:

$$\begin{cases} \frac{d}{d\xi} \Theta(\xi) = -f^d(\Theta(\xi)) - g^d(\Theta(\xi))u(\xi), \\ y(\xi) = h^d(\Theta(\xi)), \\ \Theta(0) \sim G, \end{cases} \quad (13)$$

where $u = \lambda$ is the input, y is the output, and

$$\begin{aligned} f^d(\Theta) &= \nabla_{\Theta} \mathcal{L}_{\text{data}}(\Theta, D_\Gamma), & g^d(\Theta) &= \nabla_{\Theta} \mathcal{L}_{\text{phys}}(\Theta, D_U), \\ h^d(\Theta) &= \mathcal{L}_{\text{phys}}(\Theta, D_U). \end{aligned}$$

This is a classical control-affine system with state Θ , measurement y , and control action λ . Within this framework, however, we do not take into account the resampling. A solution is to consider Monte-Carlo sampling to estimate the integrals in (5) (Goodfellow et al., 2016, Chap. 17). By the central limit theorem, we get the following equalities:

$$\begin{aligned} f^d(\Theta) &= \left\{ \frac{1}{|\Gamma|} \int_{\Gamma} \nabla_{\Theta} \|v - v_{\Theta}\|^2 \right\} + n_f \triangleq f(\Theta) + n_f, \\ g^d(\Theta) &= \left\{ \frac{1}{|U|} \int_U \nabla_{\Theta} \|\mathfrak{F}[v_{\Theta}]\|^2 \right\} + n_g \triangleq g(\Theta) + n_g, \\ h^d(\Theta) &= \left\{ \frac{1}{|U|} \int_U \|\mathfrak{F}[v_{\Theta}]\|^2 \right\} + n_h \triangleq h(\Theta) + n_h. \end{aligned}$$

where n_f, n_g, n_h are unbiased Gaussian noise with respective covariance

$$Q_f = \frac{\mathbb{V}\{\nabla_{\Theta} \|v - v_{\Theta}\|^2\}}{|D_\Gamma|}, \quad Q_g = \frac{\mathbb{V}\{\nabla_{\Theta} \|\mathfrak{F}[v_{\Theta}]\|^2\}}{|D_U|}, \quad Q_h = \frac{\mathbb{V}\{\|\mathfrak{F}[v_{\Theta}]\|^2\}}{|D_U|}.$$

Plugging this back into (13) leads to :

$$\begin{cases} \frac{d}{d\xi} \Theta(\xi) = -f(\Theta(\xi)) - g(\Theta(\xi))u(\xi) + w(\xi), \\ y(\xi) = h(\Theta(\xi)) + n(\xi), \\ \Theta(0) \sim G, \end{cases} \quad (14)$$

with unbiased disturbance w and noise n with covariances $Q_w = Q_f + uQ_g$ and Q_h . This new formulation integrates the resampling subroutine into the algorithm as stochastic elements, showing that the proposed algorithm behaves asymptotically as a stochastic differential equation. A robust and accurate algorithm for training a PINN turns into designing a controller $\xi \mapsto u(y(\xi))$ ensuring ultimate-boundedness of (14).

Remark 2. Compared to a classical control problem, both the disturbance and the noise have variances that change during training. Additionally, the disturbance variance also depends on the control input. ♦

5. CONTROLLER DESIGN

In this section, we investigate classic integral controllers for dynamical system (14).

5.1 Primal-Dual optimization / Integral controller

The Lagrangian theory (Bertsekas, 2014) rewrites discretized formulation of (5) to the following equivalent optimization problem:

$$\Theta_* \in \underset{\Theta}{\text{Argmin}} \max_{\lambda > 0} \mathcal{L}_{\lambda}^d(\Theta).$$

An approximated solution is given by the primal-dual approach, which consists of sequentially solving these two problems:

$$\Theta_*(\lambda) \in \underset{\Theta}{\text{Argmin}} \mathcal{L}_{\lambda}^d(\Theta, D_\Gamma, D_U), \quad (\text{Primal}) \quad (15)$$

and

$$\lambda_*(\Theta) \in \underset{\lambda}{\text{Argmax}} \mathcal{L}_{\lambda}^d(\Theta, D_\Gamma, D_U). \quad (\text{Dual}) \quad (16)$$

Instead of this infinite sequential training, the parallel solution, which alternates between solving the primal and the dual to get close to a local optimum of the original constrained problem, has been proposed by Goemans and Williamson (1997); Fioretto et al. (2021); Nandwani et al. (2019) in other settings.

Here, we can apply this modified algorithm, resulting in a gradient descent update of the primal problem followed by a gradient ascent on the dual to ensure that the constraints are satisfied. This leads to this formulation:

$$\begin{aligned} \Theta_{k+1} &= \Theta_k - \alpha \nabla_{\Theta} \mathcal{L}_{\lambda_k}^d(\Theta_k, D_\Gamma, D_U), & \Theta_0 &\sim G, \\ \lambda_{k+1} &= \lambda_k + k_I \alpha \nabla_{\lambda} \mathcal{L}_{\lambda_k}^d(\Theta_{k+1}, D_\Gamma, D_U), & \lambda_0 &= 0. \end{aligned}$$

Note that, in our case, we get:

$$\nabla_{\lambda} \mathcal{L}_{\lambda_k}^d(\Theta_{k+1}, D_\Gamma, D_U) = \mathcal{L}_{\text{phys}}(\Theta_{k+1}, D_U) = y(k\alpha).$$

Therefore, the dual update can be simplified to:

$$\begin{aligned} \lambda_{k+1} &= \text{compute_control}(\lambda_k, \Theta_k, D_U, D_\Theta) \\ &= \lambda_k + \alpha k_I y(k\alpha). \end{aligned}$$

Making $\alpha \rightarrow 0$ leads to $u(\xi) = \lambda(\xi) = k_I \int_0^\xi y(s) ds$, which is an integral controller for (14). Since $y, k_I \geq 0$, then $\lambda \geq 0$ and Proposition 1 still applies with a global minimum.

This interpretation of the primal-dual algorithm as a continuous-time integrator has been recently investigated by Qu and Li (2018), for instance; however, it has never been applied to the training of PINNs, which exhibits a stochastic behavior. As expected, when using an integral controller, this method ensures that no static error ($y \rightarrow 0$) occurs in the case of convergence. Under some classical assumptions (such as convexity, infinite neural network width, and small learning rates α), Theorem 2.2 by Daskalakis and Panageas (2018) mathematically states the almost surely convergence to $y = 0$, ensuring that $\mathfrak{F}[v_{\Theta}]$ is near 0.

From a control-theoretic perspective, an integral control ensures mitigation of the effects of unbiased noise and disturbances (Åström and Hägglund, 1995), along with a reduced steady-state error, which is crucial for improved accuracy and robustness.

Remark 3. (Other interpretations). Note that the integral controller could also be understood as an adversarial learning strategy, where we increase the loss (dual update) to ensure that we find the optimal parameters Θ_* in the worst case (Goodfellow et al., 2014). This is strengthening the robustness of the approach.

Another interpretation of this control method comes from curriculum learning (Bengio et al., 2009): starting from $\lambda = 0$ ensures that we first fit the measurements, then λ increases, and we start fitting the physics. Since the physics loss $\mathcal{L}_{\text{phys}}^d$ might have steep variations, it is poorly conditioned (Rathore et al., 2024) and a large λ would result in a similar conditioning for $\mathcal{L}_\lambda^d$. However, moving from a low λ to a larger one ensures that we stay in a relatively "flat" region for the physics cost and that we grow in complexity with ξ . ♦

5.2 Leaky Integral controller

It is well known that the integral controller may blow up in certain nonlinear cases, and that anti-windup strategies should be considered (Zaccarian and Teel, 2011). One solution, investigated by Weitenberg et al. (2018) is to consider an integral with a forgetting factor ϕ , written as:

$$u(\xi) = k_I \int_0^\xi y(s) \exp(-\phi(\xi - s)) ds. \quad (17)$$

This modified controller is designed to prevent boundedness of the control signal (unless y diverges exponentially fast) and will provide additional robustness through an improved stability margin. Indeed, the steady-state error might be strictly positive, and the equilibrium points will be ultimately bounded. Moreover, for a large forgetting factor ϕ , it approximates a proportional controller with probably faster convergence but no constraint on the static gain.

Differentiating Equation (17) leads to the following leaky differential equation:

$$\begin{cases} \frac{d}{d\xi} I(\xi) = -\phi I(\xi) + y(s), & I(0) = 0, \\ u(\xi) = k_I I(\xi). \end{cases} \quad (18)$$

This is a convenient form that includes the pure-integral case ($\phi = 0$) and leads to the following discretized controller:

$$\begin{aligned} \lambda_{k+1} &= \text{compute_control}(\lambda_k, \Theta_k, D_U, D_\Gamma) \\ &= (1 - \alpha k_I \phi) \lambda_k + \alpha k_I y(k\alpha). \end{aligned}$$

Note that since $y, k_I \geq 0$, then $\lambda \geq 0$ as long as $0 \leq \phi k_I \leq \alpha^{-1}$ and Proposition 1 still applies with the zero-steady state as a global minimum.

Remark 4. The leaking integrator also mitigates the effect of noise, which has a varying standard deviation. As the mean value converges, the standard deviation remains relatively constant, and the initial varying steps are discarded by the forgetting factor. ♦

5.3 Convergence properties

First, we investigate the equilibrium points and their stability before stating some practical consequences.

Local stability analysis

Proposition 3. An equilibrium point (Θ_*, I^*) of the deterministic closed-loop system satisfies

- $f(\Theta_*) = -\frac{k_I}{\phi} g(\Theta_*) h(\Theta_*)$, $I^* = \phi^{-1} h(\Theta_*)$ if $\phi > 0$;
- $f(\Theta_*) = h(\Theta_*) = 0$, $I^* \in [0, \infty)$ if $\phi = 0$.

Proof. In the case $\phi > 0$, the first equation comes straightforwardly. For the second item, one must note first that, if $\phi = 0$ then we get $h(\Theta_*) = 0$, which in turn implies $\|\mathfrak{F}[v_{\Theta_*}]\|$ is zero almost everywhere. Consequently $g(\Theta_*) = 0$ which leads to $f(\Theta_*) = 0$. □

Remark 5. If the closed-loop system converges in the case $\phi = 0$, then Θ converges to a strict subset of the equilibrium points of the system with constant $u > 0$ (vanilla PINN). ♦

Around an equilibrium point, we can linearize the system, yielding the following local stability properties for both controllers.

Proposition 4. Around an equilibrium point (Θ_*, I^*) of (14)-(18) with $\phi = 0$ where f is locally strongly convex, we get:

$$\begin{aligned} \mathbb{E}(\Theta(\xi)) &\xrightarrow[\xi \rightarrow \infty]{} \Theta_*, & \mathbb{E}(\|\Theta(\xi) - \Theta_*\|^2) &\xrightarrow[\xi \rightarrow \infty]{} \text{trace}(S), \\ \mathbb{E}(I(\xi)) &\xrightarrow[\xi \rightarrow \infty]{} I^*, & \mathbb{E}(|I(\xi) - I^*|^2) &\xrightarrow[\xi \rightarrow \infty]{} Q_h, \end{aligned}$$

where S is the solution to $S \nabla_\Theta^2 f(\Theta_*) + \nabla_\Theta^2 f(\Theta_*) S = Q_w$.

Proof. Let $z = \Theta - \Theta_*$ and $S(\xi) = \mathbb{E}(z(\xi)^\top z(\xi))$. If we start close to the equilibrium, then z is small and follows the linearized dynamics:

$$\frac{d}{d\xi} z \simeq -\nabla_\Theta^2 f(\Theta_*) z + w, \quad \frac{d}{d\xi} I \simeq \nabla_\Theta h(\Theta_*) z + n.$$

Using the same reasoning as in the previous proof, we get $\nabla_\Theta h(\Theta_*) = 0$ and therefore $\frac{d}{d\xi} I = n$. Using Itô calculus (Liu, 2019), we get:

$$\frac{d}{d\xi} S(\xi) = -S(\xi) \nabla_\Theta^2 f(\Theta_*) - \nabla_\Theta^2 f(\Theta_*) S(\xi) + Q_w.$$

Since the loss is locally strongly convex, that means that $-\nabla_\Theta^2 f(\Theta_*)$ is symmetric negative definite, then all its eigenvalues are strictly negative, ensuring that $\frac{d}{d\xi} S(\xi) \rightarrow 0$, which concludes the proof. □

In the leaky case, the following proposition holds.

Proposition 5. Around an equilibrium point $z^* = [\Theta_*^\top I^*]^\top$ of (14)-(18) and $\phi > 0$, $k_I = 1$ where $f + h(\Theta_*)\phi^{-1}h$ is locally strongly convex, then for $z = [\Theta^\top I]^\top$, we get:

$$\mathbb{E}(z(\xi)) \xrightarrow[\xi \rightarrow \infty]{} z^*, \quad \mathbb{E}(\|z(\xi) - z^*\|^2) \xrightarrow[\xi \rightarrow \infty]{} \text{trace}(S),$$

where S is the solution to $SJ + J^\top S = -\text{diag}(Q_w, Q_h)$ with

$$J = \begin{bmatrix} -\nabla_\Theta^2 f(\Theta_*) - \frac{h(\Theta_*)}{\phi} \nabla_\Theta^2 h(\Theta_*) & g(\Theta_*) \\ g(\Theta_*)^\top & -\phi \end{bmatrix}.$$

Proof. Linearizing the deterministic dynamical system around the equilibrium point z^* leads to:

$$\frac{d}{d\xi} z \simeq \begin{bmatrix} -\nabla_\Theta^2 f(\Theta_*) - \nabla_\Theta g(\Theta_*) I^* & g(\Theta_*) \\ \nabla_\Theta h(\Theta_*)^\top & -\phi \end{bmatrix} z.$$

Note that $g = \nabla_\Theta h$, therefore $\nabla_\Theta g(\Theta_*) = \nabla_\Theta^2 h(\Theta_*)$ and by using the equilibrium relationships in Proposition 4, we get $\frac{d}{d\xi} z \simeq Jz$.

The study of the stochastic differential equation follows the steps as previously. The critical part is about J being Hurwitz. Denote by $\mu(A)$ the largest real part of the eigenvalues of $A \in \mathbb{R}^{n \times n}$. Then we get

$$\begin{aligned} \mu(J) &\leq \mu\left(\frac{J + J^\top}{2}\right) \\ &\leq \max\left(\mu\left(-\nabla_{\Theta}^2 f(\Theta_*) - \frac{h(\Theta_*)}{\phi} \nabla_{\Theta}^2 h(\Theta_*)\right), -\phi\right). \end{aligned} \quad (19)$$

Since $h \geq 0$ and both $f + h(\Theta_*)\phi^{-1}h$ is strongly convex around Θ_* , then J is Hurwitz and the conclusion holds. $\square$

Connection with accuracy and robustness The accuracy (as defined in Section 2) of the closed-loop with $\phi = 0$ is improved since there are fewer equilibrium points compared to the vanilla PINN ($u > 0$ is constant), but its robustness depends only on the curvature of f and the covariances Q_w and Q_n .

A nonzero leak parameter $\phi > 0$ has two distinct and antagonistic effects on the dynamics. First, it modifies the equilibrium point z^* itself. Indeed, unless $h(\Theta_*) = 0$ one has $I^* \neq 0$ and the primal equilibrium Θ_* differs from the equilibrium of the pure-integral system. In other words, the introduction of the leak may introduce a *steady bias* in the estimator, since the controller no longer enforces the constraint $h(\Theta) = 0$ exactly at steady state. Therefore, its accuracy is decreased.

Second, increasing ϕ improves the robustness of the dynamics to noise. At the linearized level, the spectral abscissa $\mu(J)$ (from equation (19)) is nonincreasing in ϕ for low ϕ if both f and h are strongly convex around Θ_* . It follows that increasing ϕ close to 0 shifts all eigenvalues of J strictly to the left, enhancing the local damping of the dynamics. In turn, the steady-state covariance S decreases, meaning that the variance of Θ decreases as the leak increases. Moreover, the leakage ensures that the control action remains positive and lower than it would be with the pure integrator. Consequently, $\text{trace}(Q_w) = \text{trace}(Q_f) + u \text{trace}(Q_g)$ is lower asymptotically in the leaky case, decreasing even further $\text{trace}(S)$. All these effects suggest that it is enlarging the region of attraction, ensuring convergence in more scenarios.

The leak parameter ϕ , therefore, introduces a classical bias-variance trade-off: small values of ϕ enforce the physics constraint accurately but yield larger sensitivity to stochastic perturbations, while larger ϕ improves robustness at the price of a nonzero steady bias.

Remark 6. Note also that in the mismatch case with a discrepancy between the data and the physics cost, there is no stable equilibrium for the pure integrator, whereas the leaky one still allows for this possibility. $\blacklozenge$

6. NUMERICAL SIMULATIONS

In this section, we consider a toy example, and we estimate the solution to the following differential system:

$$\begin{aligned} \dot{x}(t) &= \begin{bmatrix} -5 & a \\ 0 & -2 \end{bmatrix} x(t) + \begin{bmatrix} 1 \\ 2 \end{bmatrix} (\sin(t) + \cos(t)), \quad t \in [0, 6], \\ x(0) &= [1 \ 0]^\top, \quad \Gamma_1 = \{0\}, \end{aligned} \quad (20)$$

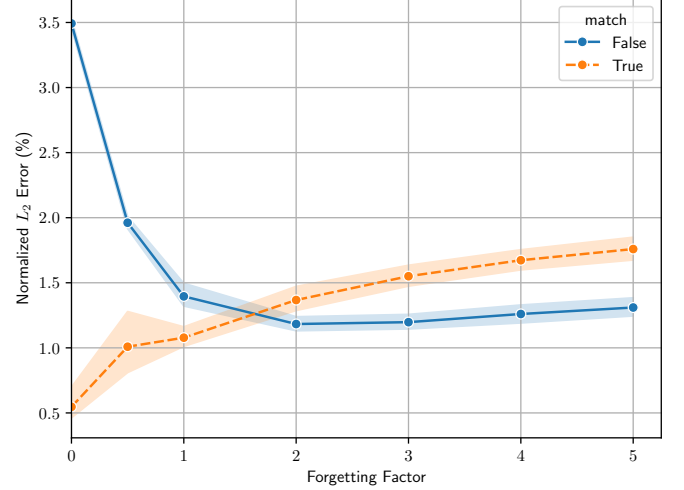


Fig. 1. Accuracy of the reconstruction as a function of the forgetting factor in the match and mismatch cases.

The parameter a accounts for model mismatch. For data-generation, we consider $a = 3$, and we measure $\hat{x}(t) = x(t) + n(t)$ at some instant $t \in \Gamma_2 = \{0.5, 1, 1.5, \dots, 6\}$ with $n \sim \mathcal{N}(0, 0.01)$.

We ran the previous algorithms with the hyperparameters from the code available on GitHub¹ and $k_I = 1$. We evaluated two cases: the *match* case, where the physics known matches the data ($a = 3$), and the *mismatch* case, where the supposed underlying model is slightly incorrect ($a = 2.7$). Since this is a toy example, we investigate whether the proposed control methodologies correctly handle these cases and compare them with the vanilla PINN. The results of 20 trials with different seeds are presented in Figure 1. The reconstructed signals are denoted $\hat{x}$ and the normalized error is defined as $e = \sqrt{\int_0^6 \|x(t) - \hat{x}(t)\|^2 dt} \cdot \|x\|_{L^2}^{-1}$.

For a zero forgetting factor, we apply the classic primal-dual methodology (pure integrator). In the matched case (dash-orange), we can see that the normalized error is very small, and the variance is also small ($0.55 \pm 0.28\%$). With vanilla PINN and a constant $\lambda = 1$, we get a normalized error of $0.61 \pm 0.46\%$, which is also good. Indeed, both of these results indicate a good fit and are likely close to the optimal one, which could be achieved by considering the approximation limitations of the finite-width neural network ($N < \infty$). We observe that the pure integrator training exhibits a lower variance than the vanilla PINN, which strengthens its robustness properties. However, in the mismatch case (solid blue), both methodologies fail to accurately estimate the real solution with $3.5 \pm 0.1\%$ normalized error for the integral control and $4.1 \pm 0.2\%$ for the vanilla PINN. Note, however, that the variance is still reduced by half when considering the integrator. This shows that the original method and the primal-dual one are comparable in terms of accuracy for that case; however, the controlled version exhibits a more robust behavior.

For the leaky integrator, in the nominal case with model match, the forgetting factor ϕ tends to deteriorate the accuracy, as evident from the increasing dash-orange curve.

¹ <https://github.com/mBarreau/Controlled-PINN>

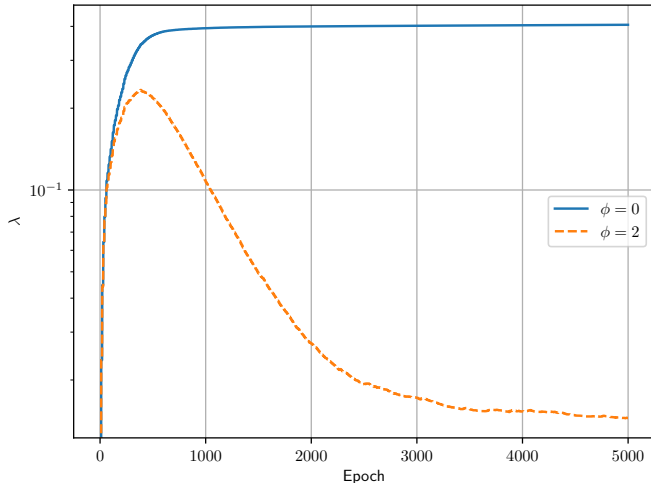


Fig. 2. Evolution of the control signal with and without integral leakage.

Unless around $\phi = 0.5$, we can see that the variance is constant. This confirms the local stability analysis results proposed previously. In the unmatched case, it is clear that the leaky integrator considerably improves performance. The normalized error decreases significantly, attaining its minimum around $\phi = 2$ with $1.2 \pm 0.13\%$. The forgetting factor provided guidance towards the physical model, but measurement fitting was more important in case of discrepancy, leading to a consistent estimation. This can be seen in Figure 2, where the control action is similar for both controllers initially, but the leaky integrator reduces as the physics loss starts to be small, ensuring good fitting with the data, while the other one continues to increase.

Overall, it appears that the integral controller indeed exhibits signs of improved robustness, which was the primary purpose of the method. On the other hand, the leaky integral introduces the classic bias-variance trade-off. A large forgetting factor does not guarantee the most accurate estimation; however, it produces very good results, even in cases of model mismatch. That means that the classical primal-dual method is appropriate for the *forward* estimation task (i.e., the problem is well-posed and there is a unique solution to (1)) while the leaky integral controller seems more appropriate for the *inverse* problem (i.e., when we try to find an underlying model behind data) since it can handle model mismatch.

7. CONCLUSION

We introduced a control-theoretic viewpoint on the training of PINNs, interpreting the learning dynamics as a feedback interconnection between the data loss and the physics residual. This perspective reveals that the primal-dual formulation corresponds to an integral controller, while adding a forgetting factor yields a leaky-integral controller.

The stability analysis shows that these two controllers exhibit different behaviors. The pure integrator guarantees exact enforcement of the physics at equilibrium, but its lack of dissipation makes it sensitive to sampling noise. In contrast, the leaky controller introduces an equilibrium bias while increasing the damping in the closed-loop

dynamics, resulting in a reduced steady-state variance. This provides a theoretical explanation for the accuracy-robustness trade-off observed experimentally on a toy example and for the superior performance of the leaky controller in mismatched or noisy settings. The integral controller performs well in the matched case, indicating that it is well-suited for forward problems where the physical model is accurate and reliable. In contrast, the leaky integrator provides more consistent results under model mismatch, indicating its potential relevance for inverse problems. In both cases, the variance of the reconstruction error is reduced compared to standard PINNs.

This work represents an initial step toward more interpretable and reliable training algorithms for PINNs. Future research will focus on extending the analysis to more complex systems and interpreting other successful PINN strategies through the lens of control theory.

REFERENCES

- Åström, K. and Hägglund, T. (1995). *PID Controllers: Theory, Design, and Tuning*. ISA - The Instrumentation, Systems and Automation Society.
- Baydin, A.G., Pearlmutter, B.A., Radul, A.A., and Siskind, J.M. (2017). Automatic differentiation in machine learning: a survey. *The Journal of Machine Learning Research*.
- Bengio, Y., Louradour, J., Collobert, R., and Weston, J. (2009). Curriculum learning. In *Proceedings of the 26th annual international conference on machine learning*.
- Bertsekas, D.P. (2014). *Constrained optimization and Lagrange multiplier methods*. Academic press.
- Cerone, V., Fosson, S.M., Pirrera, S., and Regruto, D. (2025). A new framework for constrained optimization via feedback control of lagrange multipliers. *IEEE Transactions on Automatic Control*.
- Choromanska, A., Henaff, M., Mathieu, M., Arous, G.B., and LeCun, Y. (2015). The loss surfaces of multi-layer networks. In *Artificial intelligence and statistics*. PMLR.
- Daskalakis, C. and Panageas, I. (2018). The limit points of (optimistic) gradient descent in min-max optimization. *Advances in neural information processing systems*.
- Elkabetz, O. and Cohen, N. (2021). Continuous vs. discrete optimization of deep neural networks. *Advances in Neural Information Processing Systems*.
- Fioretto, F., Van Hentenryck, P., Mak, T.W.K., Tran, C., Baldo, F., and Lombardi, M. (2021). Lagrangian duality for constrained deep learning. In *Machine Learning and Knowledge Discovery in Databases. Applied Data Science and Demo Track*. Springer International Publishing.
- Glorot, X. and Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the thirteenth international conference on artificial intelligence and statistics. JMLR Workshop and Conference Proceedings*.
- Goemans, M.X. and Williamson, D.P. (1997). The primal-dual method for approximation algorithms and its application to network design problems. *Approximation algorithms for NP-hard problems*.
- Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press.

- Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., and Bengio, Y. (2014). Generative adversarial nets. *Advances in neural information processing systems*.
- Guo, J., Yao, Y., Wang, H., and Gu, T. (2023). Pre-training strategy for solving evolution equations based on physics-informed neural networks. *Journal of Computational Physics*.
- Hartman, P. (2002). *Ordinary Differential Equations*. SIAM, Philadelphia, 2 edition. doi: 10.1137/1.9780898719222.
- Hornik, K. (1991). Approximation capabilities of multilayer feedforward networks. *Neural networks*.
- Jagtap, A.D., Kawaguchi, K., and Karniadakis, G.E. (2020). Adaptive activation functions accelerate convergence in deep and physics-informed neural networks. *Journal of Computational Physics*.
- Karniadakis, G.E., Kevrekidis, I.G., Lu, L., Perdikaris, P., Wang, S., and Yang, L. (2021). Physics-informed machine learning. *Nature Reviews Physics*.
- Kingma, D.P. and Ba, J. (2014). Adam: A method for stochastic optimization. *CoRR*.
- Ladyzhenskaia, O.A., Solonnikov, V.A., and Ural'tseva, N.N. (1968). *Linear and quasi-linear equations of parabolic type*. American Mathematical Soc.
- Li, S., Bragone, F., Barreau, M., and Morozovska, K. (2025). MILP initialization for solving parabolic PDEs with PINNs. *arXiv preprint arXiv:2501.16153*.
- Liu, K. (2019). *Stochastic stability of differential equations in abstract spaces*, volume 453. Cambridge University Press.
- Liu, Q., Chu, M., and Thuerey, N. (2024). ConFIG: Towards conflict-free training of physics informed neural networks. *arXiv preprint arXiv:2408.11104*.
- Maddu, S., Sturm, D., Müller, C.L., and Sbalzarini, I.F. (2022). Inverse Dirichlet weighting enables reliable training of physics-informed neural networks. *Machine Learning: Science and Technology*.
- McClenny, L.D. and Braga-Neto, U.M. (2023). Self-adaptive physics-informed neural networks. *Journal of Computational Physics*, 474, 111722.
- Nandwani, Y., Pathak, A., and Singla, P. (2019). A primal dual formulation for deep learning with constraints. In H.M. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E.B. Fox, and R. Garnett (eds.), *Advances in Neural Information Processing Systems (NeurIPS)*.
- Psaros, A.F., Kawaguchi, K., and Karniadakis, G.E. (2022). Meta-learning PINN loss functions. *Journal of computational physics*.
- Qu, G. and Li, N. (2018). On the exponential stability of primal-dual gradient dynamics. *IEEE Control Systems Letters*.
- Rathore, P., Lei, W., Frangella, Z., Lu, L., and Udell, M. (2024). Challenges in training pinns: a loss landscape perspective. In *Proceedings of the 41st International Conference on Machine Learning, ICML'24*. JMLR.org.
- Shukla, K., Jagtap, A.D., and Karniadakis, G.E. (2021). Parallel physics-informed neural networks via domain decomposition. *Journal of Computational Physics*.
- Sirignano, J. and Spiliopoulos, K. (2018). DGM: A deep learning algorithm for solving partial differential equations. *Journal of computational physics*.
- Son, H., Jang, J.W., Han, W.J., and Hwang, H.J. (2023). Sobolev training for physics informed neural networks. *Communications in Mathematical Sciences*.
- Wang, S., Teng, Y., and Perdikaris, P. (2021). Understanding and mitigating gradient pathologies in physics-informed neural networks. *SIAM Journal on Scientific Computing*.
- Wang, S., Yu, X., and Perdikaris, P. (2022). When and why PINNs fail to train: A neural tangent kernel perspective. *Journal of Computational Physics*.
- Weitenberg, E., Jiang, Y., Zhao, C., Mallada, E., De Persis, C., and Dörfler, F. (2018). Robust decentralized secondary frequency control in power systems: Merits and tradeoffs. *IEEE Transactions on Automatic Control*, 64(10), 3967–3982.
- Zaccarian, L. and Teel, A.R. (2011). *Modern anti-windup synthesis: control augmentation for actuator saturation*. Princeton University Press.